

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



VIEWMODEL DAN DEBUGGING

Oleh:

Ashma Salimah

NIM. 2410817220015

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel dan Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Ashma Salimah
NIM : 2410817220015

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code	6
B. Output Program.....	48
C. Pembahasan.....	50
SOAL 2	50
A. Jawaban.....	60
TAUTAN GITHUB	62

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Test Debugging Compose Step Over	48
Gambar 2. Screenshot Hasil Test Debugging Compose Step Into	48
Gambar 3. Screenshot Hasil Test Debugging Compose Step Out	49
Gambar 4. Screenshot Hasil Test Debugging XML Step Over	49
Gambar 5. Screenshot Hasil Test Debugging XML Step Into	50
Gambar 6. Screenshot Hasil Test Debugging XML Step Out	50

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt Compose	6
Tabel 2. Source Code Anime.kt Compose.....	9
Tabel 3. Source Code AnimeData.kt Compose	9
Tabel 4. Source Code AnimeCarousel.kt Compose.....	11
Tabel 5. Source Code AnimeList.kt Compose.....	14
Tabel 6. Source Code DetailScreen.kt Compose	17
Tabel 7. Source Code SettingScreen.kt Compose.....	20
Tabel 8. Source Code AnimeViewModel.kt Compose.....	22
Tabel 9. Source Code MainActivity.kt XML	24
Tabel 10. Source Code Anime.kt XML	24
Tabel 11. Source Code AnimeData.kt XML	25
Tabel 12. Source Code HomeFragment.kt XML	27
Tabel 13. Source Code DetailFragment.kt XML	29
Tabel 14. Source Code SettingsFragment.kt XML	30
Tabel 15. Source Code AnimeViewModel.kt XML	32
Tabel 16. Source Code AnimeAdapter.kt XML	33
Tabel 17. Source Code CarouselAdapter.kt XML	35
Tabel 18. Source Code CarouselWrapperAdapter.kt XML	36
Tabel 19. Source Code activity_main.xml.....	38
Tabel 20. Source Code fragment_home.xml	39
Tabel 21. Source Code fragment_detail.xml	40
Tabel 22. Source Code fragment_setting.xml.....	42
Tabel 23. Source Code item_anime.xml.....	43
Tabel 24. Source Code item_carousel.xml	45
Tabel 25. Source Code item_carousel_wrapper.xml	47

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada

Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
 - d. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

Tabel 1. Source Code MainActivity.kt Compose

No	MainActivity.kt (Compose)
1	package com.example.composeModul3
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.activity.viewModels
8	import androidx.compose.foundation.layout.PaddingValues
9	import androidx.compose.foundation.lazy.LazyColumn
10	import androidx.compose.material.icons.Icons

11	import
	androidx.compose.material3.ExperimentalMaterial3Api
12	import androidx.compose.material3.Icon
13	import androidx.compose.material3.IconButton
14	import androidx.compose.material3.Scaffold
15	import androidx.compose.material3.Text
16	import androidx.compose.material3.TopAppBar
17	import androidx.compose.runtime.Composable
18	import androidx.compose.ui.text.font.FontWeight
19	import androidx.navigation.NavHostController
20	import androidx.navigation.compose.NavHost
21	import androidx.navigation.compose.composable
22	import
	androidx.navigation.compose.rememberNavController
23	import com.example.composeModul3.data. <i>animeList</i>
24	import
	com.example.composeModul3.ui.components.AnimeList
25	import com.example.composeModul3.ui.theme.COMPOSETheme
26	import androidx.compose.foundation.lazy.items
27	import
	com.example.composeModul3.ui.components.AnimeCarousel
28	import
	com.example.composeModul3.ui.screen.DetailsScreen
29	import
	com.example.composeModul3.ui.screen.SettingScreen
30	import androidx.compose.material.icons.filled. <i>Language</i>
31	import androidx.compose.ui.res.stringResource
32	import
	com.example.composeModul3.viewmodel.AnimeViewModel
33	import
	com.example.composeModul3.viewmodel.AnimeViewModelFacto ry
34	
35	class MainActivity : ComponentActivity() {
36	private val viewModel: AnimeViewModel by
	viewModels() {
37	AnimeViewModelFactory("MuhammadFulan")
38	}
39	
40	override fun onCreate(savedInstanceState: Bundle?)
	{
41	super.onCreate(savedInstanceState)
42	enableEdgeToEdge()
43	setContent {

```

44         COMPOSETheme(darkTheme = true) {
45             MyApp(viewModel = viewModel)
46         }
47     }
48 }
49 }
50
51 @Composable
52 fun MyApp(viewModel: AnimeViewModel) {
53     val navController = rememberNavController()
54     NavHost(navController, startDestination = "main") {
55         composable("main") { MainScreen(navController,
56             viewModel) }
57         composable("details/{title}") { backStackEntry
58             ->
59                 val title =
60                 backStackEntry.arguments?.getString("title") ?: ""
61                 DetailsScreen(title = title, navController)
62             }
63         composable("setting") {
64             SettingScreen(navController) }
65     }
66 }
67
68 @OptIn(ExperimentalMaterial3Api::class)
69 @Composable
70 fun MainScreen(navController: NavHostController,
71     viewModel: AnimeViewModel) {
72     Scaffold(
73         topBar = {
74             TopAppBar(
75                 title = {
76                 Text(stringResource(R.string.app_name), fontWeight =
77                 FontWeight.Bold) },
78                 actions = {
79                 IconButton(onClick = {
80                 navController.navigate("setting") }) {
81                 Icon(Icons.Default.Language,
82                 contentDescription = "settings")
83                 }
84                 }
85             )
86         }
87     ) { innerPadding ->
88         Main(paddingValues = innerPadding,

```


80	navController = navController, viewModel = viewModel)
81	}
82	}
83	@Composable
84	fun Main(paddingValues: PaddingValues, navController: NavHostController, viewModel: AnimeViewModel) {
85	LazyColumn(contentPadding = paddingValues) {
86	item {
87	AnimeCarousel(animeList = animeList)
88	}
89	items(animeList) { anime ->
90	AnimeList(anime = anime, navController = navController, viewModel = viewModel)
91	}
92	}
93	}

Tabel 2. Source Code Anime.kt Compose

No	Anime.kt [model] (Compose)
1	package com.example.composeModul3.model
2	
3	data class Anime(
4	val title: String,
5	val premiered: String,
6	val description: Int,
7	val descLong: Int,
8	val image: Int,
9	val imageBg: Int,
10	val webUrl: String
11	)

Tabel 3. Source Code AnimeData.kt Compose

No	AnimeData.kt [data] (Compose)
1	package com.example.composeModul3.data
2	
3	import com.example.composeModul3.R
4	import com.example.composeModul3.model.Anime
5	
6	val animeList = listOf(
7	Anime(
8	title = "Jujutsu Kaisen",

9	premiered = "Fall 2020",
10	description = R.string.desc_jjk,
11	descLong = R.string.desc_long_jjk,
12	image = R.drawable.jjk,
13	imageBg = R.drawable.jjk_bg,
14	webView =
	"https://www.crunchyroll.com/series/GRDV0019R/jujutsu-
	kaisen"
15	),
16	Anime(
17	title = "Mashle: Magic and Muscles",
18	premiered = "Spring 2023",
19	description = R.string.desc_mashle,
20	descLong = R.string.desc_long_mashle,
21	image = R.drawable.mashle,
22	imageBg = R.drawable.mashle_bg,
23	webView =
	"https://www.crunchyroll.com/series/GDKHZEP8W/mashle-
	magic-and-muscles"
24	),
25	Anime(
26	title = "Kuroshitsuji",
27	premiered = "Fall 2008",
28	description = R.string.desc_kuro,
29	descLong = R.string.desc_long_kuro,
30	image = R.drawable.kuro,
31	imageBg = R.drawable.kuro_bg,
32	webView =
	"https://www.crunchyroll.com/series/GYQ43P3E6/black-
	butler"
33	),
34	Anime(
35	title = "Kamonohashi Ron no Kindan Suiri",
36	premiered = "Fall 2023",
37	description = R.string.desc_ron,
38	descLong = R.string.desc_long_ron,
39	image = R.drawable.ron,
40	imageBg = R.drawable.ron_bg,
41	webView =
	"https://www.crunchyroll.com/series/GNVHKN903/ron-
	kamonohashis-forbidden-deductions"
42	),
43	Anime(
44	title = "Kimetsu No Yaiba",
45	premiered = "Spring 2019",

46	description = R.string.desc_kny,
47	descLong = R.string.desc_long_kny,
48	image = R.drawable.kny,
49	imageBg = R.drawable.kny_bg,
50	webView =
	"https://www.crunchyroll.com/series/GY5P48XEY/demon-slayer-kimetsu-no-yaiba"
51	),
52	Anime(
53	title = "Ganbare! Nakamura-kun!!",
54	premiered = "-",
55	description = R.string.desc_nakamura,
56	descLong = R.string.desc_long_nakamura,
57	image = R.drawable.nakamura,
58	imageBg = R.drawable.nakamura_bg,
59	webView =
	"https://www.crunchyroll.com/series/GT00337506/go-for-it-nakamura-kun"
60	),
61	Anime(
62	title = "Boku no Hero Academia",
63	premiered = "Spring 2016",
64	description = R.string.desc_bnha,
65	descLong = R.string.desc_long_bnha,
66	image = R.drawable.bnha,
67	imageBg = R.drawable.bnha_bg,
68	webView =
	"https://www.crunchyroll.com/series/G6NQ5DWZ6/my-hero-academia"
69	)
70	)

Tabel 4. Source Code AnimeCarousel.kt Compose

No	AnimeCarousel.kt [ui] (Compose)
1	package com.example.composeModul3.ui.components
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import
	androidx.compose.foundation.pager.HorizontalPager
6	import
	androidx.compose.foundation.pager.rememberPagerState
7	import

```

androidx.compose.foundation.shape.RoundedCornerShape
 8 import androidx.compose.material3.*
 9 import androidx.compose.runtime.Composable
10 import androidx.compose.ui.Alignment
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.graphics.Color
13 import androidx.compose.ui.layout.ContentScale
14 import androidx.compose.ui.res.painterResource
15 import androidx.compose.ui.res.stringResource
16 import androidx.compose.ui.text.font.FontWeight
17 import androidx.compose.ui.unit.dp
18 import com.example.composeModul3.model.Anime
19 import androidx.compose.ui.res.stringResource
20 import com.example.composeModul3.R
21
22 @Composable
23 fun AnimeCarousel(animeList: List<Anime>) {
24
25     val pagerState = rememberPagerState(
26         initialState = 0,
27         pageCount = { Int.MAX_VALUE })
28
29     Column(modifier = Modifier.padding(vertical =
30 8.dp)) {
31         Text(
32             text =
33             (stringResource(R.string.featured)),
34             style =
35             MaterialTheme.typography.titleMedium,
36             fontWeight = FontWeight.Bold,
37             modifier = Modifier.padding(horizontal =
38 16.dp, vertical = 4.dp))
39
40         HorizontalPager(
41             state = pagerState,
42             contentPadding = PaddingValues(horizontal
43 = 32.dp),
44             pageSpacing = 12.dp,
45             modifier = Modifier
46                 .fillMaxWidth()
47                 .height(220.dp)) {
48             page ->
49                 val anime = animeList[page %
50 animeList.size]
51                 CarouselItem(anime = anime)

```

```

46         }
47
48         Row(
49             modifier = Modifier
50                 .fillMaxWidth()
51                 .padding(top = 8.dp),
52             horizontalArrangement =
Arrangement.Center) {
53             repeat(animeList.size) { index ->
54                 val isSelected =
pagerState.currentPage % animeList.size == index
55                 Box(
56                     modifier = Modifier
57                         .padding(horizontal = 4.dp)
58                         .size(if (isSelected) 8.dp
else 6.dp)
59                     .then(
60                         Modifier.padding(0.dp)),
61                     contentAlignment =
Alignment.Center) {
62                     Surface(
63                         modifier =
Modifier.fillMaxSize(),
64                         shape =
RoundedCornerShape(50),
65                         color = if (isSelected)
66                             MaterialTheme.colorScheme.primary
67                             else
68                             MaterialTheme.colorScheme.onSurface.copy(alpha =
0.3f)) {}
69                     }
70                 }
71             }
72         }
73     }
74
75     @Composable
76     fun CarouselItem(anime: Anime) {
77         Card(
78             modifier = Modifier
79                 .fillMaxWidth()
80                 .height(200.dp),
81             shape = RoundedCornerShape(12.dp)) {

```

82	Box(modifier = Modifier. <i>fillMaxSize</i> ()) {
83	Image(
84	painter = painterResource(id =
	anime.imageBg),
85	contentDescription = anime.title,
86	contentScale = ContentScale.Crop,
87	modifier = Modifier. <i>fillMaxSize</i> ())
88	Box(
89	modifier = Modifier
90	. <i>fillMaxWidth</i> ()
91	.align(Alignment.BottomCenter)
92	.padding(12.dp)) {
93	Column {
94	Text(
95	text = anime.title,
96	style =
	MaterialTheme.typography.bodyMedium,
97	fontWeight = FontWeight.Bold,
98	color = Color.White,
99	maxLines = 2)
100	Text(
101	text = anime.premiered,
102	style =
	MaterialTheme.typography.labelSmall,
103	color = Color.White)
104	}
105	}
106	}
107	}
108	}

Tabel 5. Source Code AnimeList.kt Compose

No	AnimeList.kt [ui] (Compose)
1	package com.example.composeModul3.ui.components
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.layout.Arrangement
7	import androidx.compose.foundation.layout.Column
8	import androidx.compose.foundation.layout.Row
9	import androidx.compose.foundation.layout.Spacer
10	import
	androidx.compose.foundation.layout.fillMaxHeight

```

11 import androidx.compose.foundation.layout.fillMaxWidth
12 import androidx.compose.foundation.layout.height
13 import androidx.compose.foundation.layout.padding
14 import androidx.compose.foundation.layout.width
15 import
    androidx.compose.foundation.shape.RoundedCornerShape
16 import androidx.compose.material3.Card
17 import androidx.compose.material3.MaterialTheme
18 import androidx.compose.material3.OutlinedButton
19 import androidx.compose.material3.Text
20 import androidx.compose.runtime.Composable
21 import androidx.compose.ui.Modifier
22 import androidx.compose.ui.draw.clip
23 import androidx.compose.ui.layout.ContentScale
24 import androidx.compose.ui.platform.LocalContext
25 import androidx.compose.ui.res.painterResource
26 import androidx.compose.ui.res.stringResource
27 import androidx.compose.ui.text.font.FontWeight
28 import androidx.compose.ui.unit.dp
29 import androidx.navigation.NavHostController
30 import
    com.example.composeModul3.viewmodel.AnimeViewModel
31 import com.example.composeModul3.R
32 import com.example.composeModul3.model.Anime
33 import timber.log.Timber
34
35 @Composable
36 fun AnimeList(anime: Anime, navController:
    NavHostController, viewModel: AnimeViewModel) {
37     val context = LocalContext.current
38
39     Card(
40         modifier = Modifier
41             .fillMaxWidth()
42             .padding(horizontal = 16.dp, vertical = 8.dp),
43         shape = RoundedCornerShape(12.dp)) {
44         Row(modifier = Modifier.padding(16.dp)) {
45             Image(
46                 painter = painterResource(id =
    anime.image),
47                 contentDescription = "",
48                 contentScale = ContentScale.Crop,
49                 modifier = Modifier
50                     .fillMaxHeight()
51                     .clip(RoundedCornerShape(8.dp)))

```

52	
53	Spacer(modifier = Modifier.width(8.dp))
54	
55	Column(modifier = Modifier.fillMaxWidth())
	{
56	Row(modifier = Modifier
	.fillMaxWidth(),
57	horizontalArrangement =
	Arrangement.SpaceBetween) {
58	Text(
59	text = anime.title,
60	style =
	MaterialTheme.typography.titleMedium,
61	fontWeight = FontWeight.Bold,
62	modifier =
	Modifier.weight(1f))
63	Text(
64	text = anime.premiered,
65	style =
	MaterialTheme.typography.bodySmall)
66	}
67	
68	Spacer(modifier =
	Modifier.height(4.dp))
69	
70	Text(
71	text =
	(stringResource(anime.description)),
72	style =
	MaterialTheme.typography.bodySmall,
73	maxLines = 4)
74	
75	Spacer(modifier =
	Modifier.height(8.dp))
76	
77	Row(horizontalArrangement =
	Arrangement.spacedBy(8.dp)) {
78	OutlinedButton(
79	onClick = {
80	Timber.d("Navigating to
	\${anime.webUrl}")
81	val intent =
	Intent(Intent.ACTION_VIEW) .apply {
82	data =
	Uri.parse(anime.webUrl)

83	setPackage("com.crunchyroll.crunchyroid")
84	}
85	if(intent.resolveActivity(context.packageManager) != null) {
86	context.startActivity(intent)
87	} else {
88	val fallback = Intent(Intent.ACTION_VIEW, Uri.parse(anime.webUrl))
89	context.startActivity(fallback)
90	}
91	},
92	shape = RoundedCornerShape(12.dp)) {
93	Text(stringResource(R.string.watch))
94	}
95	
96	OutlinedButton(97 onClick = { 98 Timber.d("Navigating to details screen for \${anime.title}")
99	navController.navigate("details/\${anime.title}") },
100	shape = RoundedCornerShape(12.dp)) {
101	Text(stringResource(R.string.detail))
102	}
103	}
104	}
105	}
106	}
107	}

Tabel 6. Source Code DetailScreen.kt Compose

No	DetailScreen.kt [screen] (Compose)
1	package com.example.composeModul3.ui.screen
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.background

```

5 import androidx.compose.foundation.layout.*
6 import
  androidx.compose.foundation.shape.RoundedCornerShape
7 import androidx.compose.material3.*
8 import androidx.compose.runtime.Composable
9 import androidx.compose.ui.Modifier
10 import androidx.compose.ui.graphics.Color
11 import androidx.compose.ui.layout.ContentScale
12 import androidx.compose.ui.res.painterResource
13 import androidx.compose.ui.res.stringResource
14 import androidx.compose.ui.text.font.FontWeight
15 import androidx.compose.ui.unit.dp
16 import com.example.composeModul3.data.animeList
17 import androidx.navigation.NavHostController
18 import com.example.composeModul3.R
19 import timber.log.Timber
20
21 @Composable
22 fun DetailsScreen(title: String, navController:
  NavHostController) {
23     val anime = animeList.find { it.title == title }
24
25     Timber.d("Navigating to details screen for anime
  $title")
26
27     if (anime == null) {
28         Text("Anime not found")
29         return
30     }
31
32     Column(modifier =
  Modifier.fillMaxSize().background(MaterialTheme.colors
  scheme.background)) {
33         Image(
34             painter = painterResource(id =
  anime.imageBg),
35             contentDescription = anime.title,
36             contentScale = ContentScale.Crop,
37             modifier = Modifier
38                 .fillMaxWidth()
39                 .height(300.dp))
40
41         Column(modifier = Modifier.padding(16.dp)) {
42             Row(

```

```

43             modifier = Modifier.fillMaxWidth(),
44             horizontalArrangement =
Arrangement.SpaceBetween) {
45                 Text(
46                     text = anime.title,
47                     style =
MaterialTheme.typography.headlineSmall,
48                     fontWeight = FontWeight.Bold,
49                     color =
MaterialTheme.colorScheme.onBackground,
50                     modifier = Modifier.weight(1f))
51                 Text(
52                     text = anime.premiered,
53                     style =
MaterialTheme.typography.bodyMedium,
54                     color =
MaterialTheme.colorScheme.onBackground,)
55             }
56
57             Spacer(modifier = Modifier.height(12.dp))
58
59             Text(text =
stringResource(R.string.synopsis),
60                 style =
MaterialTheme.typography.titleMedium,
61                 fontWeight = FontWeight.Bold,
62                 color =
MaterialTheme.colorScheme.onBackground,)
63
64             Spacer(modifier = Modifier.height(4.dp))
65
66             Text(
67                 text =
(stringResource(anime.descLong)),
68                 style =
MaterialTheme.typography.bodyMedium,
69                 color =
MaterialTheme.colorScheme.onBackground)
70
71             Spacer(modifier = Modifier.height(16.dp))
72
73             Button(
74                 onClick = { navController.navigateUp()
},

```

75	modifier = Modifier.fillMaxWidth(),
76	shape = RoundedCornerShape(8.dp),
77	colors =
	ButtonDefaults.outlinedButtonColors(
78	contentColor = Color(0xFF2D1B45),
79	containerColor =
	Color(0xFFB39DDB))) {
80	Text(text =
	stringResource(R.string.back))
81	}
82	}
83	}
84	}

Tabel 7. Source Code SettingScreen.kt Compose

No	SettingScreen.kt [screen] (Compose)
1	package com.example.composeModul3.ui.screen
2	
3	import android.app.LocaleManager
4	import android.content.Context
5	import android.os.Build
6	import android.os.LocaleList
7	import androidx.appcompat.app.AppCompatActivityDelegate
8	import androidx.compose.foundation.layout.*
9	import
	androidx.compose.foundation.shape.RoundedCornerShape
10	import androidx.compose.material3.*
11	import androidx.compose.runtime.Composable
12	import androidx.compose.ui.Alignment
13	import androidx.compose.ui.Modifier
14	import androidx.compose.ui.graphics.Color
15	import androidx.compose.ui.platform.LocalContext
16	import androidx.compose.ui.res.stringResource
17	import androidx.compose.ui.text.font.FontWeight
18	import androidx.compose.ui.unit.dp
19	import androidx.core.os.LocaleListCompat
20	import com.example.composeModul3.R
21	import androidx.navigation.NavHostController
22	
23	@OptIn(ExperimentalMaterial3Api::class)
24	@Composable
25	fun SettingScreen(navController: NavHostController) {
26	val context = LocalContext.current

```

27
28 Scaffold(
29     topBar = {
30         TopAppBar(
31             title = {
32
33                 Text(
34                     text =
35                     stringResource(R.string.settings_title),
36                     fontWeight = FontWeight.Bold)
37             })
38         }) { innerPadding ->
39         Column(
40             modifier = Modifier
41                 .fillMaxSize()
42                 .padding(innerPadding)
43                 .padding(16.dp),
44             verticalArrangement =
45             Arrangement.spacedBy(12.dp)) {
46             Text(
47                 text =
48                 stringResource(R.string.select_language),
49                 style =
50                 MaterialTheme.typography.titleMedium,
51                 fontWeight = FontWeight.Bold)
52
53             Button(
54                 onClick = { setLocale(context, "en")
55             },
56                 modifier = Modifier.fillMaxWidth()) {
57                 Text(text =
58                 stringResource(R.string.english))
59             }
60
61             Button(
62                 onClick = { setLocale(context, "in")},
63                 modifier = Modifier.fillMaxWidth()) {
64                 Text(text =
65                 stringResource(R.string.indonesian))
66             }
67
68             Button(
69                 onClick = { navController.navigateUp()
70             },

```

62	modifier = Modifier.fillMaxWidth(),
63	shape = RoundedCornerShape(8.dp),
64	colors =
	ButtonDefaults.outlinedButtonColors(
65	contentColor = Color(0xFF2D1B45),
66	containerColor =
	Color(0xFFB39DDB))) {
67	Text(text =
	stringResource(R.string.back))
68	}
69	}
70	}
71	}
72	
73	fun setLocale(context: Context, language: String) {
74	if (Build.VERSION.SDK_INT >=
	Build.VERSION_CODES.TIRAMISU) {
75	context.getSystemService(LocaleManager::class.java)
76	.applicationLocales =
	LocaleList.forLanguageTags(language)
77	} else {
78	AppCompatDelegate.setApplicationLocales(
79	LocaleListCompat.forLanguageTags(language)
80	)
81	}
82	}

Tabel 8. Source Code AnimeViewModel.kt Compose

No	AnimeViewModel.kt (Compose)
1	package com.example.composeModul3.viewmodel
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.composeModul3.data.animeList as
	animeDataList
6	import com.example.composeModul3.model.Anime
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.StateFlow
9	import kotlinx.coroutines.flow.asStateFlow
10	import timber.log.Timber
11	
12	class AnimeViewModel(private val username: String) :

```

13     ViewModel() {
14         private val _animeList =
15             MutableStateFlow(animeDataList)
16         val animeList: StateFlow<List<Anime>> =
17             _animeList.asStateFlow()
18
19         private val _selectedAnime =
20             MutableStateFlow<String?>(null)
21         val selectedAnime: StateFlow<String?> =
22             _selectedAnime.asStateFlow()
23
24         private val _toUrl =
25             MutableStateFlow<String?>(null)
26         val toUrl: StateFlow<String?> =
27             _toUrl.asStateFlow()
28
29         init {
30             Timber.d("User : $username - loading anime
31 list...")
32             Timber.d("Anime list loaded:
33 ${_animeList.value.size} items")
34         }
35
36         fun onAnimeSelected(title: String) {
37             Timber.d("Anime selected: $title")
38             _selectedAnime.value = title
39         }
40
41         fun clearSelection() {
42             _selectedAnime.value = null
43         }
44
45         fun onLinkClicked(url: String) {
46             Timber.d("Link clicked: $url")
47             _toUrl.value = url
48         }
49
50         fun onUrlNavigated() {
51             _toUrl.value = null
52         }
53     }
54
55     class AnimeViewModelFactory(private val username:
56 String) : ViewModelProvider.Factory{
57         override fun <T : ViewModel> create(modelClass:

```

48	Class<T>): T {
	if
	(modelClass.isAssignableFrom(AnimeViewModel::class.java
	a)) {
49	@Suppress("UNCHECKED_CAST")
50	return AnimeViewModel(username) as T
51	}
52	throw IllegalArgumentException("Unknown
	ViewModel class")
53	}
54	}

Tabel 9. Source Code MainActivity.kt XML

No	MainActivity.kt (XML)
1	package com.example.xmlModul3
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import
	com.example.xmlModul3.databinding.ActivityMainBinding
6	
7	class MainActivity : AppCompatActivity() {
8	private lateinit var binding: ActivityMainBinding
9	
10	override fun onCreate(savedInstanceState: Bundle?) {
11	super.onCreate(savedInstanceState)
12	binding =
	ActivityMainBinding.inflate(layoutInflater)
13	setContentView(binding.root)
14	}
15	}

Tabel 10. Source Code Anime.kt XML

No	Anime.kt [model] (XML)
1	package com.example.animexml.model
2	
3	data class Anime(
4	val title: String,
5	val premiered: String,
6	val description: Int,
7	val descLong: Int,
8	val image: Int,

9	val imageBg: Int,
10	val webUrl: String
11	)

Tabel 11. Source Code AnimeData.kt XML

No	AnimeData.kt [data] (XML)
1	package com.example.xmlModul3.data
2	
3	import com.example.animexml.model.Anime
4	import com.example.xmlModul3.R
5	
6	val animeList = listOf(
7	Anime(
8	title = "Jujutsu Kaisen",
9	premiered = "Fall 2020",
10	description = R.string.desc_jjk,
11	descLong = R.string.desc_long_jjk,
12	image = R.drawable.jjk,
13	imageBg = R.drawable.jjk_bg,
14	webUrl =
	"https://www.crunchyroll.com/series/GRDV0019R/jujutsu-
	kaisen"
15	
16	),
17	Anime(
18	title = "Mashle: Magic and Muscles",
19	premiered = "Spring 2023",
20	description = R.string.desc_mashle,
21	descLong = R.string.desc_long_mashle,
22	image = R.drawable.mashle,
23	imageBg = R.drawable.mashle_bg,
	webUrl =
	"https://www.crunchyroll.com/series/GDKHZEP8W/mashle-
24	magic-and-muscles"
25	),
26	Anime(
27	title = "Kuroshitsuji",
28	premiered = "Fall 2008",
29	description = R.string.desc_kuro,
30	descLong = R.string.desc_long_kuro,
31	image = R.drawable.kuro,
32	imageBg = R.drawable.kuro_bg,
	webUrl =

33	"https://www.crunchyroll.com/series/GYQ43P3E6/black-butler"
34	),
35	Anime(
36	title = "Kamonohashi Ron no Kindan Suiri",
37	premiered = "Fall 2023",
38	description = R.string.desc_ron,
39	descLong = R.string.desc_long_ron,
40	image = R.drawable.ron,
41	imageBg = R.drawable.ron_bg,
	webViewUrl =
42	"https://www.crunchyroll.com/series/GNVHKN903/ron-kamonohashis-forbidden-deductions"
43	),
44	Anime(
45	title = "Kimetsu No Yaiba",
46	premiered = "Spring 2019",
47	description = R.string.desc_kny,
48	descLong = R.string.desc_long_kny,
49	image = R.drawable.kny,
50	imageBg = R.drawable.kny_bg,
	webViewUrl =
51	"https://www.crunchyroll.com/series/GY5P48XEY/demon-slayer-kimetsu-no-yaiba"
52	),
53	Anime(
54	title = "Ganbare! Nakamura-kun!!",
55	premiered = "-",
56	description = R.string.desc_nakamura,
57	descLong = R.string.desc_long_nakamura,
58	image = R.drawable.nakamura,
59	imageBg = R.drawable.nakamura_bg,
	webViewUrl =
60	"https://www.crunchyroll.com/series/GT00337506/go-for-it-nakamura-kun"
61	),
62	Anime(
63	title = "Boku no Hero Academia",
64	premiered = "Spring 2016",
65	description = R.string.desc_bnha,
66	descLong = R.string.desc_long_bnha,
67	image = R.drawable.bnha,
68	imageBg = R.drawable.bnha_bg,
	webViewUrl =
	"https://www.crunchyroll.com/series/G6NQ5DWZ6/my-hero-

69	academia"
70	)

Tabel 12. Source Code HomeFragment.kt XML

No	HomeFragment.kt [fragment] (XML)
1	package com.example.xmlModul3.ui.fragment
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.fragment.app.viewModels
9	import androidx.navigation.fragment.findNavController
10	import androidx.recyclerview.widget.ConcatAdapter
11	import androidx.recyclerview.widget.LinearLayoutManager
12	import com.example.xmlModul3.AnimeViewModel
13	import com.example.xmlModul3.AnimeViewModelFactory
14	import com.example.xmlModul3.R
15	import com.example.xmlModul3.adapter.AnimeAdapter
16	import com.example.xmlModul3.adapter.CarouselAdapter
17	import
	com.example.xmlModul3.adapter.CarouselWrapperAdapter
18	import
	com.example.xmlModul3.databinding.FragmentHomeBinding
19	import timber.log.Timber
20	
21	class HomeFragment : Fragment() {
22	private var _binding: FragmentHomeBinding? = null
23	private val binding get() = _binding!!
24	private val viewModel: AnimeViewModel by viewModels
	{
25	AnimeViewModelFactory("MuhammadFulan")
26	}
27	
28	override fun onCreateView(
29	inflater: LayoutInflater, container: ViewGroup?,
30	savedInstanceState: Bundle?
31	): View {
32	_binding = FragmentHomeBinding.inflate(inflater,
	container, false)
33	return binding.root
34	}

```

35
36     override fun onCreateView(view: View,
37 savedInstanceState: Bundle?) {
38         super.onCreateView(view, savedInstanceState)
39         setupToolbar()
40         setupRecyclerView()
41     }
42     private fun setupToolbar() {
43         binding.toolbar.title =
44         getString(R.string.app_name)
45         binding.toolbar.inflateMenu(R.menu.menu_home)
46         binding.toolbar.setOnMenuItemClickListener {
47             menuItem ->
48                 when (menuItem.itemId) {
49                     R.id.action_language -> {
50                         findNavController().navigate(R.id.action_home_to_setting
51 )
52                         true
53                     }
54                     else -> false
55                 }
56             }
57         }
58     private fun setupRecyclerView() {
59         val carouselAdapter =
60         CarouselAdapter(viewModel.animeList.value)
61
62         val animeAdapter =
63         AnimeAdapter(viewModel.animeList.value) { anime ->
64             Timber.d("Anime selected: ${anime.title}")
65             val bundle = Bundle().apply {
66                 putString("title", anime.title)
67             }
68             findNavController().navigate(R.id.action_home_to_detail,
69 bundle)
70         }
71
72         val concatAdapter = ConcatAdapter(
73             CarouselWrapperAdapter(
74                 carouselAdapter = carouselAdapter,
75                 itemCount =

```

71	viewModel.animeList.value.size,
72	onPageChanged = { _: Int -> }
73	),
74	animeAdapter
75	)
76	binding.recyclerView.layoutManager =
77	LinearLayoutManager(requireContext())
78	binding.recyclerView.adapter = concatAdapter
79	}
80	override fun onDestroyView() {
81	super.onDestroyView()
82	_binding = null
83	}
84	}

Tabel 13. Source Code DetailFragment.kt XML

No	DetailFragment.kt [fragment] (XML)
1	package com.example.xmlModul3.ui.fragment
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.navigation.fragment.findNavController
9	import com.bumptech.glide.Glide
10	import com.example.xmlModul3.data.animeList
11	import
12	com.example.xmlModul3.databinding.FragmentDetailBinding
13	import timber.log.Timber
14	class DetailFragment : Fragment() {
15	private var _binding: FragmentDetailBinding? = null
16	private val binding get() = _binding!!
17	
18	override fun onCreateView(
19	inflater: LayoutInflater, container: ViewGroup?,
20	savedInstanceState: Bundle?
21	): View {
22	_binding =
	FragmentDetailBinding.inflate(inflater, container,

	false)
23	return binding.root
24	}
25	
26	override fun onCreateView(view: View,
	savedInstanceState: Bundle?) {
27	super.onCreateView(view, savedInstanceState)
28	
29	val title = arguments?.getString("title") ?: ""
30	val anime = animeList.find { it.title == title }
31	
32	Timber.d("Navigating to details screen for
	\$title")
33	
34	anime?.let {
35	binding.textTitle.text = it.title
36	binding.textPremiered.text = it.premiered
37	binding.textDescription.text =
	getString(it.descLong)
38	
39	Glide.with(this)
40	.load(it.imageBg)
41	.centerCrop()
42	.into(binding.imageBg)
43	}
44	
45	binding.buttonBack.setOnClickListener {
46	Timber.d("Navigating to details screen for
	`\${title}`")
47	findNavController().navigateUp()
48	}
49	}
50	
51	override fun onDestroyView() {
52	super.onDestroyView()
53	_binding = null
54	}
55	}

Tabel 14. Source Code SettingsFragment.kt XML

No	SettingsFragment.kt [fragment] (XML)
1	package com.example.xmlModul3.ui.fragment
2	
3	import android.app.LocaleManager

```

4 import android.content.Context
5 import android.os.Build
6 import android.os.Bundle
7 import android.os.LocaleList
8 import android.view.LayoutInflater
9 import android.view.View
10 import android.view.ViewGroup
11 import androidx.appcompat.app.AppCompatActivity
12 import androidx.core.os.LocaleListCompat
13 import androidx.fragment.app.Fragment
14 import androidx.navigation.fragment.findNavController
15 import
com.example.xmlModul3.databinding.FragmentSettingBinding
16
17 class SettingFragment : Fragment() {
18     private var _binding: FragmentSettingBinding? = null
19     private val binding get() = _binding!!
20
21     override fun onCreateView(
22         inflater: LayoutInflater, container: ViewGroup?,
23         savedInstanceState: Bundle?
24     ): View {
25         _binding =
FragmentSettingBinding.inflate(inflater, container,
false)
26         return binding.root
27     }
28
29     override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
30         super.onViewCreated(view, savedInstanceState)
31
32         binding.buttonEnglish.setOnClickListener {
33             setLocale(requireContext(), "en")
34         }
35
36         binding.buttonIndonesian.setOnClickListener {
37             setLocale(requireContext(), "in")
38         }
39
40         binding.buttonBack.setOnClickListener {
41             findNavController().navigateUp()
42         }
43     }

```

44	
45	private fun setLocale(context: Context, language: String) {
46	if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
47	android.util.Log.d("SettingFragment", "Using LocaleManager")
48	context.getSystemService(LocaleManager::class.java)
49	.applicationLocales = LocaleList.forLanguageTags(language)
50	} else {
51	android.util.Log.d("SettingFragment", "Using AppCompatActivity")
52	AppCompatActivity.setApplicationLocales(
53	LocaleListCompat.forLanguageTags(language)
54	)
55	}
56	requireActivity().recreate()
57	}
58	
59	override fun onDestroyView() {
60	super.onDestroyView()
61	_binding = null
62	}
63	}

Tabel 15. Source Code AnimeViewModel.kt XML

No	AnimeViewModel.xml
1	package com.example.xmlModul3
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.animexml.model.Anime
6	import com.example.xmlModul3.data.animeList as animeDataList
7	import kotlinx.coroutines.flow.MutableStateFlow
8	import kotlinx.coroutines.flow.StateFlow
9	import kotlinx.coroutines.flow.asStateFlow
10	import timber.log.Timber
11	
12	class AnimeViewModel(private val username: String) : ViewModel() {

13	private val _animeList =
	MutableStateFlow(animeDataList)
14	val animeList: StateFlow<List<Anime>> =
	_animeList.asStateFlow()
15	
16	init {
17	Timber.d("User : \$username - loading anime
	list")
18	Timber.d("Anime list loaded:
	\${_animeList.value.size} items")
19	}
20	}
21	
22	class AnimeViewModelFactory(private val username:
	String) : ViewModelProvider.Factory {
23	override fun <T : ViewModel> create(modelClass:
	Class<T>): T {
24	if
	(modelClass.isAssignableFrom(AnimeViewModel::class.java)
	) {
25	@Suppress("UNCHECKED_CAST")
26	return AnimeViewModel(username) as T
27	}
28	throw IllegalArgumentException("Unknown
	ViewModel class")
29	}
30	}

Tabel 16. Source Code AnimeAdapter.kt XML

No	AnimeAdapter.kt [adapter] (XML)
1	package com.example.xmlModul3.adapter
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.view.LayoutInflater
6	import android.view.ViewGroup
7	import androidx.recyclerview.widget.RecyclerView
8	import com.bumptech.glide.Glide
9	import com.example.animexml.model.Anime
10	import
	com.example.xmlModul3.databinding.ItemAnimeBinding
11	import timber.log.Timber

```

12
13 class AnimeAdapter(
14     private val animeList: List<Anime>,
15     private val onDetailClick: (Anime) -> Unit
16 ) : RecyclerView.Adapter<AnimeAdapter.AnimeViewHolder>()
17 {
18     override fun onCreateViewHolder(parent: ViewGroup,
19     viewType: Int): AnimeViewHolder {
20         val binding = ItemAnimeBinding.inflate(
21             LayoutInflater.from(parent.context), parent,
22             false
23         )
24         return AnimeViewHolder(binding)
25     }
26
27     override fun onBindViewHolder(holder:
28     AnimeViewHolder, position: Int) {
29         holder.bind(animeList[position])
30     }
31
32     override fun getItemCount() = animeList.size
33
34     inner class AnimeViewHolder(private val binding:
35     ItemAnimeBinding) :
36         RecyclerView.ViewHolder(binding.root) {
37
38         fun bind(anime: Anime) {
39             binding.textTitle.text = anime.title
40             binding.textPremiered.text = anime.premiered
41             binding.textDescription.text =
42             itemView.context.getString(anime.description)
43
44             Glide.with(binding.root)
45                 .load(anime.image)
46                 .centerCrop()
47                 .into(binding.imageAnime)
48
49             binding.buttonWatch.setOnClickListener {
50                 Timber.d("Navigating to ${anime.title}")
51                 val intent =
52                 Intent(Intent.ACTION_VIEW).apply {
53                     data = Uri.parse(anime.webUrl)
54                 }
55                 startActivity(intent)
56             }
57         }
58     }
59
60     setPackage("com.crunchyroll.crunchyroid")

```

49	}
50	if
	(intent.resolveActivity(itemView.context.packageManager)
	!= null) {
51	itemView.context.startActivity(intent)
52	} else {
53	val fallback =
	Intent(Intent.ACTION_VIEW, Uri.parse(anime.webUrl))
54	itemView.context.startActivity(fallback)
55	}
56	}
57	
58	binding.buttonDetail.setOnClickListener {
59	onDetailClick(anime)
60	}
61	}
62	}
63	}

Tabel 17. Source Code CarouselAdapter.kt XML

No	CarouselAdapter.kt (XML)
1	package com.example.xmlModul3.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.bumptech.glide.Glide
7	import com.example.animexml.model.Anime
8	import
9	com.example.xmlModul3.databinding.ItemCarouselBinding
10	
	class CarouselAdapter(private val animeList:
	List<Anime>) :
11	
	RecyclerView.Adapter<CarouselAdapter.CarouselViewHolder>
12	() {
13	
14	override fun getItemCount() = Int.MAX_VALUE
15	
	override fun onCreateViewHolder(parent: ViewGroup,
16	viewType: Int): CarouselViewHolder {
17	val binding = ItemCarouselBinding.inflate(

	LayoutInflater.from(parent.context), parent,
	false
18	)
19	return CarouselViewHolder(binding)
20	}
21	
22	override fun onBindViewHolder(holder:
	CarouselViewHolder, position: Int) {
23	holder.bind(animeList[position %
	animeList.size])
24	}
25	
26	inner class CarouselViewHolder(private val binding:
	ItemCarouselBinding) :
27	RecyclerView.ViewHolder(binding.root) {
28	fun bind(anime: Anime) {
29	binding.textCarouselTitle.text = anime.title
30	binding.textCarouselPremiered.text =
	anime.premiered
31	Glide.with(binding.root)
32	.load(anime.imageBg)
33	.centerCrop()
34	.into(binding.imageCarousel)
35	}
36	}
37	}

Tabel 18. Source Code CarouselWrapperAdapter.kt XML

No	CarouselWrapperAdapter.kt [adapter] (XML)
1	package com.example.xmlModul3.adapter
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import android.widget.ImageView
6	import androidx.recyclerview.widget.RecyclerView
7	import androidx.viewpager2.widget.ViewPager2
8	import com.example.xmlModul3.R
9	import
	com.example.xmlModul3.databinding.ItemCarouselWrapperBin
	ding
10	
11	class CarouselWrapperAdapter(
12	private val carouselAdapter: CarouselAdapter,
13	private val itemCount: Int,

```

14     private val onPageChanged: (Int) -> Unit
15 ) :
    RecyclerView.Adapter<CarouselWrapperAdapter.ViewHolder>(
    ) {
16
17     override fun onCreateViewHolder(parent: ViewGroup,
    viewType: Int): ViewHolder {
18         val binding =
    ItemCarouselWrapperBinding.inflate(
19             LayoutInflater.from(parent.context), parent,
    false
20         )
21         return ViewHolder(binding)
22     }
23
24     override fun onBindViewHolder(holder: ViewHolder,
    position: Int) {
25         holder.bind()
26     }
27
28     override fun getItemCount() = 1
29
30     inner class ViewHolder(private val binding:
    ItemCarouselWrapperBinding) :
31         RecyclerView.ViewHolder(binding.root) {
32
33         fun bind() {
34             binding.viewPager.adapter = carouselAdapter
35             setupDots()
36
37             binding.viewPager.registerOnPageChangeCallback(object :
38                 ViewPager2.OnPageChangeCallback() {
39                 override fun onPageSelected(position:
    Int) {
40                     onPageChanged(position)
41
42                     updateDots(position % itemCount)
43                 }
44             })
45
46             private fun setupDots() {
47                 binding.dotIndicator.removeAllViews()

```

48	repeat(itemCount) { index ->
49	val dot =
50	ImageView(binding.root.context).apply {
51	setImageResource(
52	if (index == 0)
53	R.drawable.dot_selected
54	else R.drawable.dot_unselected
55	)
56	val params =
57	ViewGroup.MarginLayoutParams(16, 16)
58	params.setMargins(8, 0, 8, 0)
59	layoutParams = params
60	}
61	binding.dotIndicator.addView(dot)
62	}
63	}
64	private fun updateDots(selectedIndex: Int) {
65	for (i in 0 until
66	binding.dotIndicator.childCount) {
67	val dot =
68	binding.dotIndicator.getChildAt(i) as ImageView
69	dot.setImageResource(
70	if (i == selectedIndex)
71	R.drawable.dot_selected
72	else R.drawable.dot_unselected
	)
	}
	}
	}

Tabel 19. Source Code activity_main.xml

No	activity_main.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:fitsSystemWindows="true">

8	<androidx.fragment.app.FragmentContainerView
9	android:id="@+id/nav_host_fragment"
10	
11	android:name="androidx.navigation.fragment.NavHostFragment"
12	android:layout_width="match_parent"
13	android:layout_height="match_parent"
14	app:defaultNavHost="true"
15	app:navGraph="@navigation/nav_graph"
16	app:layout_constraintTop_toTopOf="parent"
17	app:layout_constraintBottom_toBottomOf="parent"/>
18	
19	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 20. Source Code fragment_home.xml

No	fragment_home.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:layout_width="match_parent"
4	android:layout_height="match_parent"
5	android:orientation="vertical">
6	
7	<androidx.appcompat.widget.Toolbar
8	android:id="@+id/toolbar"
9	android:layout_width="match_parent"
10	android:layout_height="?attr/actionBarSize"
11	android:background="?attr/colorSurface"
12	android:elevation="4dp"/>
13	
14	<LinearLayout
15	android:id="@+id/dotIndicator"
16	android:layout_width="match_parent"
17	android:layout_height="wrap_content"
18	android:gravity="center"
19	android:orientation="horizontal"
20	android:padding="8dp"
21	android:visibility="gone"/>
22	
23	<androidx.recyclerview.widget.RecyclerView
24	android:id="@+id/recyclerView"

25	android:layout_width="match_parent"
26	android:layout_height="0dp"
27	android:layout_weight="1"
28	android:padding="8dp"/>
29	
30	</LinearLayout>

Tabel 21. Source Code fragment_detail.xml

No	fragment_detail.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:layout_width="match_parent"
4	android:layout_height="match_parent">
5	
6	<LinearLayout
7	android:layout_width="match_parent"
8	android:layout_height="wrap_content"
9	android:orientation="vertical">
10	
11	<ImageView
12	android:id="@+id/imageBg"
13	android:layout_width="match_parent"
14	android:layout_height="300dp"
15	android:scaleType="centerCrop" />
16	
17	<LinearLayout
18	android:layout_width="match_parent"
19	android:layout_height="wrap_content"
20	android:orientation="vertical"
21	android:padding="16dp">
22	
23	<LinearLayout
24	android:layout_width="match_parent"
25	android:layout_height="wrap_content"
26	android:orientation="horizontal">
27	
28	<TextView
29	android:id="@+id/textTitle"
30	android:layout_width="0dp"
31	android:layout_height="wrap_content"
32	android:layout_weight="1"

33	android:textAppearance="?attr/textAppearanceHeadlineSmall"
34	android:textStyle="bold"/>
35	
36	<TextView
37	android:id="@+id/textPremiered"
38	android:layout_width="wrap_content"
39	android:layout_height="wrap_content"
40	android:textAppearance="?attr/textAppearanceBodyMedium"/>
41	
42	</LinearLayout>
43	
44	<View
45	android:layout_width="match_parent"
46	android:layout_height="12dp"/>
47	
48	<TextView
49	android:layout_width="wrap_content"
50	android:layout_height="wrap_content"
51	android:text="@string/synopsis"
52	android:textAppearance="?attr/textAppearanceTitleMedium"
53	android:textStyle="bold"/>
54	
55	<View
56	android:layout_width="match_parent"
57	android:layout_height="4dp"/>
58	
59	<TextView
60	android:id="@+id/textDescription"
61	android:layout_width="match_parent"
62	android:layout_height="wrap_content"
63	android:textAppearance="?attr/textAppearanceBodyMedium"/>
64	
65	<View
66	android:layout_width="match_parent"
67	android:layout_height="12dp"/>

68	
69	<Button
70	android:id="@+id/buttonBack"
71	android:layout_width="match_parent"
72	android:layout_height="wrap_content"
73	android:text="@string/back"/>
74	
75	</LinearLayout>
76	</LinearLayout>
77	</ScrollView>

Tabel 22. Source Code fragment_setting.xml

No	fragment_setting.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	android:layout_width="match_parent"
4	android:layout_height="match_parent"
5	android:orientation="vertical"
6	android:padding="16dp">
7	
8	<androidx.appcompat.widget.Toolbar
9	android:id="@+id/toolbar"
10	android:layout_width="match_parent"
11	android:layout_height="?attr/actionBarSize"
12	android:background="?attr/colorSurface"/>
13	
14	<TextView
15	android:layout_width="wrap_content"
16	android:layout_height="wrap_content"
17	android:text="@string/select_language"
18	android:textAppearance="?attr/textAppearanceTitleMedium"
19	android:textStyle="bold"
20	android:paddingBottom="12dp"/>
21	
22	<Button
23	android:id="@+id/buttonEnglish"
24	android:layout_width="match_parent"
25	android:layout_height="wrap_content"
26	android:text="@string/english"/>

27	
28	<Button
29	android:id="@+id/buttonIndonesian"
30	android:layout_width="match_parent"
31	android:layout_height="wrap_content"
32	android:text="@string/indonesian"
33	android:layout_marginTop="8dp"/>
34	
35	<Button
36	android:id="@+id/buttonBack"
37	android:layout_width="match_parent"
38	android:layout_height="wrap_content"
39	android:text="@string/back"
40	android:layout_marginTop="8dp"/>
41	
42	</LinearLayout>

Tabel 23. Source Code item_anime.xml

No	item_anime.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_margin="8dp"
8	app:cardCornerRadius="12dp"
9	app:cardElevation="4dp">
10	
11	<LinearLayout
12	android:layout_width="match_parent"
13	android:layout_height="wrap_content"
14	android:orientation="horizontal"
15	android:padding="12dp">
16	
17	<ImageView
18	android:id="@+id/imageAnime"
19	android:layout_width="100dp"
20	android:layout_height="match_parent"
21	android:scaleType="centerCrop"/>

```

22
23         <LinearLayout
24             android:layout_width="match_parent"
25             android:layout_height="wrap_content"
26             android:orientation="vertical"
27             android:layout_marginStart="12dp">
28
29             <LinearLayout
30                 android:layout_width="match_parent"
31                 android:layout_height="wrap_content"
32                 android:orientation="horizontal">
33
34                 <TextView
35                     android:id="@+id/textTitle"
36                     android:layout_width="0dp"
37                     android:layout_height="wrap_content"
38                     android:layout_weight="1"
39                     android:textStyle="bold"
40                     android:textAppearance="?attr/textAppearanceTitleMedium"/>
41
42                     <TextView
43                         android:id="@+id/textPremiered"
44                         android:layout_width="wrap_content"
45                         android:layout_height="wrap_content"
46                         android:textAppearance="?attr/textAppearanceBodySmall"/>
47
48                 </LinearLayout>
49
50                 <TextView
51                     android:id="@+id/textDescription"
52                     android:layout_width="match_parent"
53                     android:layout_height="wrap_content"
54                     android:maxLines="3"
55                     android:layout_marginTop="4dp"
56                     android:textAppearance="?attr/textAppearanceBodySmall"/>
57

```

58	<LinearLayout
59	android:layout_width="match_parent"
60	android:layout_height="wrap_content"
61	android:orientation="horizontal"
62	android:layout_marginTop="8dp">
63	
	<com.google.android.material.button.MaterialButton
64	android:id="@+id/buttonWatch"
65	
	android:layout_width="wrap_content"
66	
	android:layout_height="wrap_content"
67	android:text="@string/watch"
68	app:cornerRadius="12dp"
69	
	style="@style/Widget.Material3.Button.OutlinedButton"/
70	>
	<com.google.android.material.button.MaterialButton
71	android:id="@+id/buttonDetail"
72	
73	
	android:layout_width="wrap_content"
74	
	android:layout_height="wrap_content"
75	android:text="@string/detail"
76	android:layout_marginStart="8dp"
77	app:cornerRadius="12dp"
78	
	style="@style/Widget.Material3.Button.OutlinedButton"/
79	>
80	</LinearLayout>
81	</LinearLayout>
82	</LinearLayout>
83	
84	</com.google.android.material.card.MaterialCardView>
85	

Tabel 24. Source Code item_carousel.xml

No	item_carousel.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView

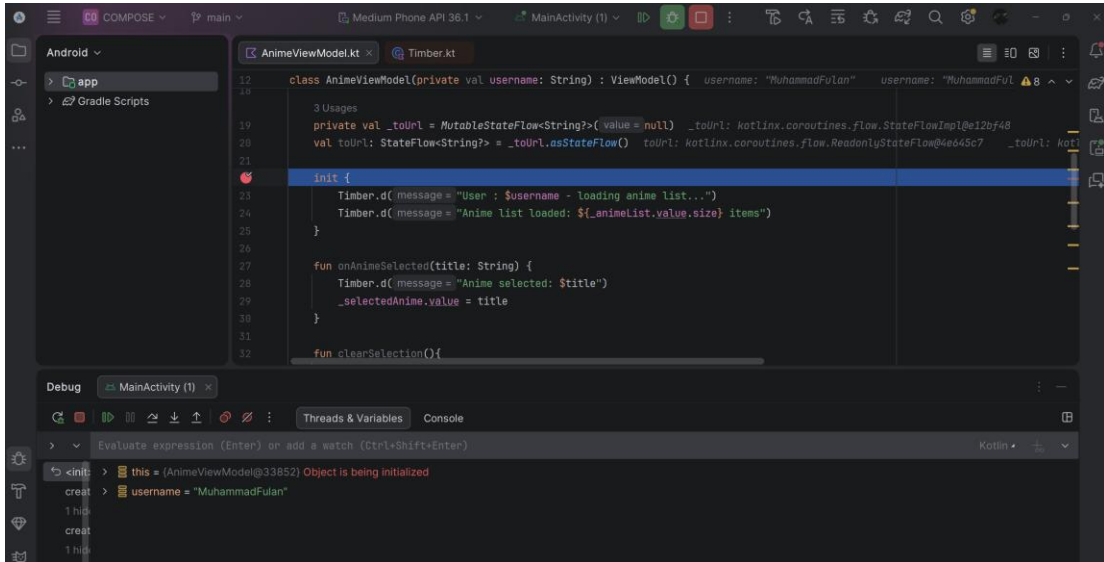
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:layout_margin="8dp"
8	app:cardCornerRadius="12dp">
9	
10	<FrameLayout
11	android:layout_width="match_parent"
12	android:layout_height="match_parent">
13	
14	<ImageView
15	android:id="@+id/imageCarousel"
16	android:layout_width="match_parent"
17	android:layout_height="match_parent"
18	android:scaleType="centerCrop"/>
19	
20	<LinearLayout
21	android:layout_width="match_parent"
22	android:layout_height="wrap_content"
23	android:layout_gravity="bottom"
24	android:orientation="vertical"
25	android:padding="12dp"
26	android:background="#80000000">
27	
28	<TextView
29	android:id="@+id/textCarouselTitle"
30	android:layout_width="match_parent"
31	android:layout_height="wrap_content"
32	android:textColor="@android:color/white"
33	android:textStyle="bold"
34	android:maxLines="2"
35	android:textAppearance="?attr/textAppearanceBodyMedium"/>
36	
37	<TextView
38	android:id="@+id/textCarouselPremiered"
39	android:layout_width="match_parent"
40	android:layout_height="wrap_content"

41	android:textColor="@android:color/white"
42	android:textAppearance="?attr/textAppearanceLabelSmall" />
43	
44	</LinearLayout>
45	</FrameLayout>
46	
47	</com.google.android.material.card.MaterialCardView>

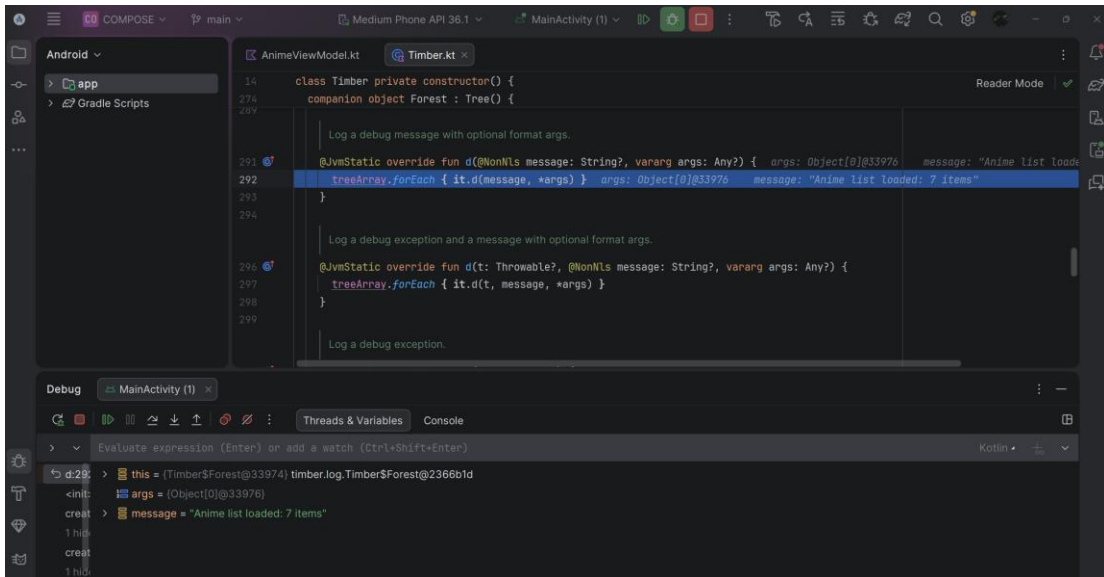
Tabel 25. Source Code item_carousel_wrapper.xml

No	item_carousel_wrapper.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	android:layout_width="match_parent"
5	android:layout_height="wrap_content"
6	android:orientation="vertical">
7	<androidx.viewpager2.widget.ViewPager2
8	android:id="@+id/viewPager"
9	android:layout_width="match_parent"
10	android:layout_height="220dp" />
11	
12	<LinearLayout
13	android:id="@+id/dotIndicator"
14	android:layout_width="match_parent"
15	android:layout_height="wrap_content"
16	android:gravity="center"
17	android:orientation="horizontal"
18	android:padding="8dp" />
19	
20	</LinearLayout>

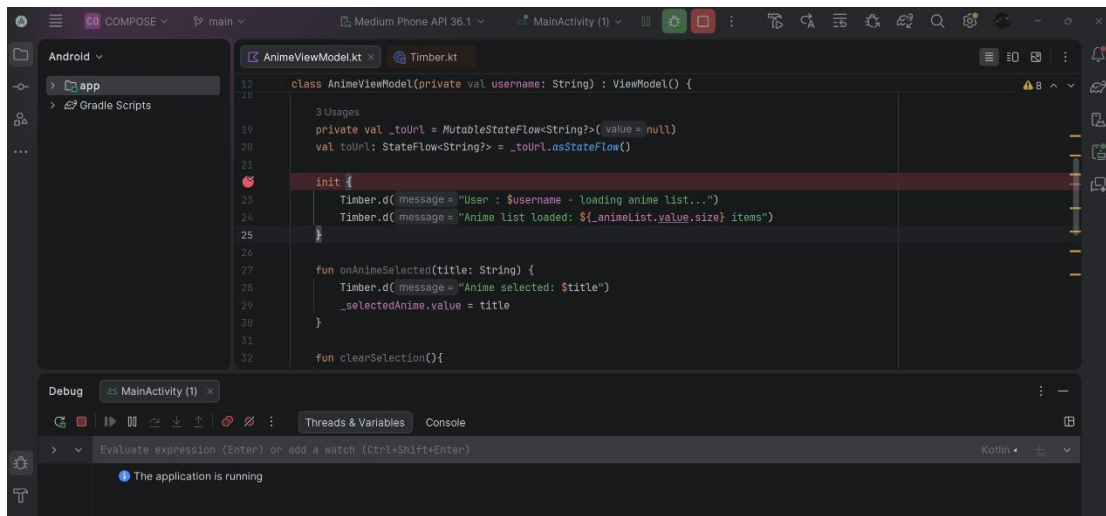
B. Output Program



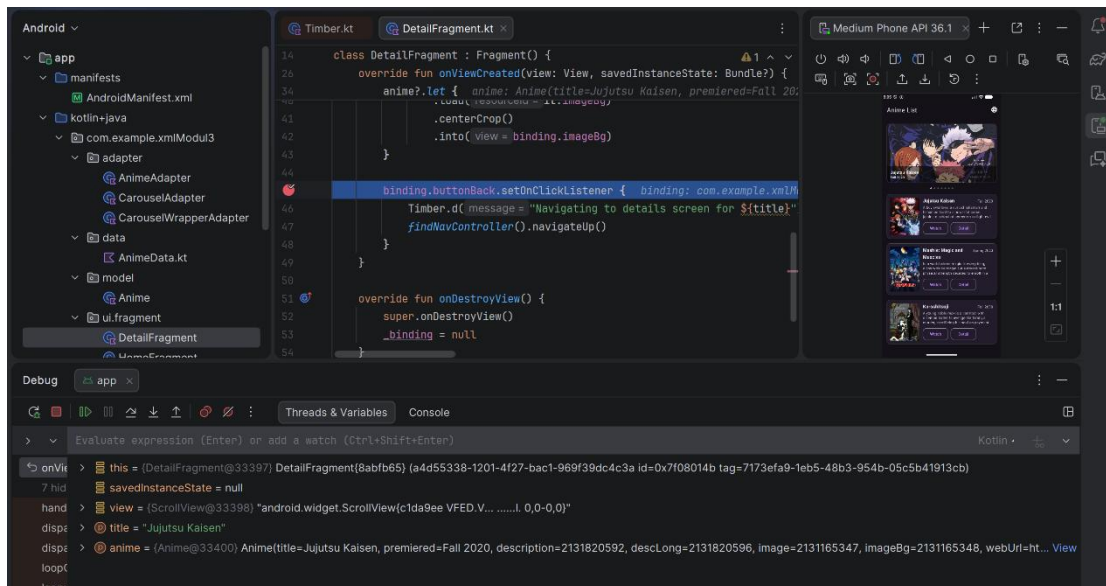
Gambar 1. Screenshot Hasil Test Debugging Compose Step Over



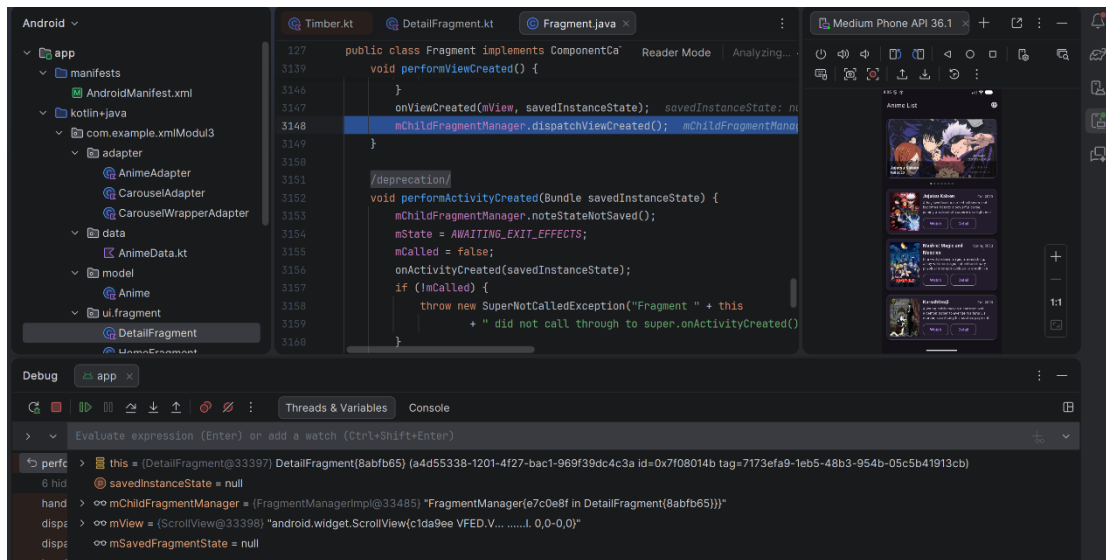
Gambar 2. Screenshot Hasil Test Debugging Compose Step Into



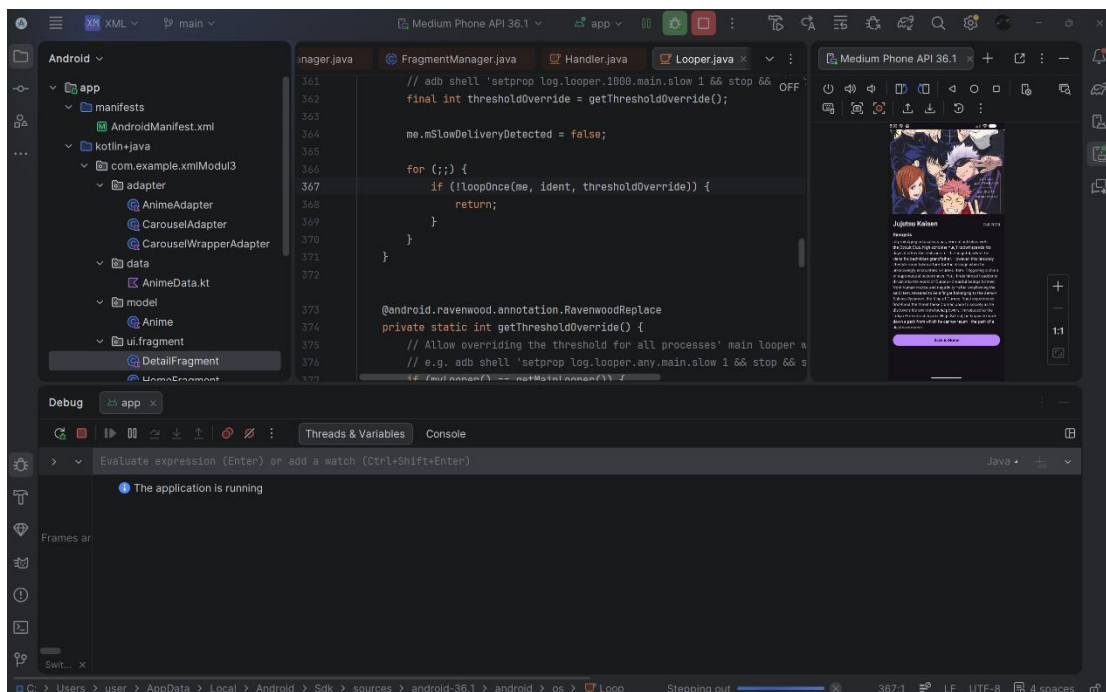
Gambar 3. Screenshot Hasil Test Debugging Compose Step Out



Gambar 4. Screenshot Hasil Test Debugging XML Step Over



Gambar 5. Screenshot Hasil Test Debugging XML Step Into



Gambar 6. Screenshot Hasil Test Debugging XML Step Out

C. Pembahasan

MainActivity.kt (Compose)

File ini adalah titik masuk utama aplikasi Compose. Berisi inisialisasi AnimeViewModel menggunakan AnimeViewModelFactory dengan parameter

username, sehingga ViewModel bisa menerima data awal saat dibuat. Di dalamnya juga terdapat fungsi MyApp() yang mengatur sistem navigasi menggunakan NavHost. MyApp() menentukan layar mana yang ditampilkan berdasarkan rute navigasi. MainScreen() menampilkan TopAppBar dengan ikon bahasa dan konten utama berupa LazyColumn, sedangkan Main() menyusun daftar anime menggunakan AnimeCarousel di bagian atas dan AnimeList untuk setiap item anime di bawahnya.

Anime.kt (Compose)

File ini adalah data class yang menjadi blueprint struktur data setiap objek anime. Setiap objek Anime memiliki properti yaitu, title, premiered, description dan descLong, image dan imageBg, dan webUrl. Karena menggunakan data class, Kotlin otomatis men-generate fungsi seperti equals(), hashCode(), dan copy().

AnimeData.kt (Compose)

File ini berfungsi sebagai sumber data statis aplikasi. Berisi satu list bernama animeList yang menampung tujuh objek Anime dengan data yang sudah di hardcode. Karena menggunakan listOf(), data ini bersifat immutable atau tidak bisa diubah. List ini menjadi sumber data utama yang digunakan oleh AnimeViewModel untuk diekspos ke UI.

AnimeViewModel.kt (Compose)

File ini mengimplementasikan pola MVVM dan menjadi inti pengelolaan data aplikasi. AnimeViewModel menerima parameter username bertipe String melalui konstruktor, sehingga membutuhkan AnimeViewModelFactory untuk membuatnya. Di dalamnya terdapat tiga StateFlow yaitu, _animeList untuk menyimpan daftar anime, _selectedAnime untuk menyimpan judul anime yang dipilih, dan _toUrl untuk menyimpan URL yang akan dibuka di browser.

Pada blok init, Timber mencatat dua log saat data pertama kali dimuat yaitu informasi username dan jumlah item yang berhasil dimuat. Fungsi onAnimeSelected() mencatat log judul anime yang dipilih dan memperbarui nilai _selectedAnime. Fungsi

clearSelection() mereset pilihan anime kembali ke null. Fungsi onLinkClicked() mencatat log URL yang diklik dan menyimpannya ke _toUrl. Fungsi onUrlNavigated() mereset nilai _toUrl menjadi null setelah URL berhasil dibuka di browser.

AnimeViewModelFactory adalah kelas pembantu yang mengimplementasikan ViewModelProvider.Factory. Fungsinya adalah membuat instance AnimeViewModel dengan parameter username yang tidak bisa dilakukan oleh sistem Android secara default. Fungsi create() mengecek apakah ViewModel yang diminta adalah AnimeViewModel, jika iya maka dibuat dengan username yang sudah disiapkan, jika tidak maka melempar IllegalArgumentException.

App.kt (Compose)

File ini adalah Application Class yang pertama kali dijalankan sebelum komponen lain seperti Activity. Di dalamnya dilakukan inisialisasi Timber dengan Timber.plant(Timber.DebugTree()) agar library Timber siap digunakan di seluruh aplikasi. Tanpa file ini, semua Timber.d() di kode tidak akan muncul di Logcat. File ini didaftarkan di AndroidManifest.xml dengan atribut android:name=".App".

AnimeCarousel.kt (Compose)

File ini menampilkan carousel gambar anime secara horizontal di bagian atas layar utama. Menggunakan HorizontalPager dari Compose Foundation dengan pageCount = Int.MAX_VALUE untuk menciptakan efek infinite scroll, dimana saat pengguna geser terus, carousel tidak akan berhenti. Modulo digunakan saat mengambil data agar index tidak melebihi jumlah data. Di bawah carousel terdapat dot indicator yang menunjukkan posisi halaman saat ini.

AnimeList.kt (Compose)

File ini adalah komponen kartu untuk setiap item anime di daftar. Menerima parameter anime, NavController, dan viewModel. Setiap kartu menampilkan gambar, judul, tahun tayang, deskripsi singkat, dan dua tombol aksi. Tombol Watch mencatat log Timber dengan pesan URL yang diklik, lalu membuka aplikasi Crunchyroll terlebih dahulu

menggunakan Intent dengan `setPackage()`. Jika Crunchyroll tidak terinstall maka fallback ke browser. Tombol Detail mencatat log Timber dengan pesan judul anime yang dipilih, lalu menavigasi ke halaman detail menggunakan `navController.navigate()`.

DetailScreen.kt (Compose)

File ini adalah layar detail yang menampilkan informasi lengkap sebuah anime. Menerima parameter title dari navigasi, lalu mencari data anime yang cocok menggunakan fungsi `find{}` dari Kotlin Collection. Saat pertama kali ditampilkan, Timber mencatat log judul anime yang sedang dibuka. Menampilkan gambar, judul, tahun tayang, sinopsis panjang, dan tombol kembali. Jika anime tidak ditemukan, ditampilkan teks "Anime not found".

SettingScreen.kt (Compose)

File ini menangani fitur ganti bahasa aplikasi secara runtime. Menggunakan dua pendekatan berbeda tergantung versi Android yaitu, `LocaleManager` untuk Android 13 ke atas, dan `AppCompatDelegate` untuk versi lebih lama. Strategi dua jalur ini memastikan fitur ganti bahasa berjalan di semua versi Android yang didukung. Terdapat dua tombol bahasa, yaitu bahasa Inggris dan Indonesia dan satu tombol kembali.

MainActivity.kt (XML)

File ini adalah titik masuk utama aplikasi XML. Berbeda dengan versi Compose, `MainActivity` di versi XML sangat tipis, hanya bertugas meng-inflate layout `activity_main.xml` menggunakan View Binding dan menyetel content view. Semua logika navigasi dan UI diserahkan sepenuhnya ke Fragment. Ini adalah pola umum dalam arsitektur XML Android yang memisahkan tanggung jawab antara Activity dan Fragment.

Anime.kt (XML)

File ini identik dengan versi Compose. Merupakan data class yang mendefinisikan struktur objek Anime dengan properti title, premiered, description, descLong, image, imageBg, dan webUrl. Semua field description, image, dan imageBg bertipe Int karena menyimpan ID resource Android, bukan nilai langsung.

AnimeData.kt (XML)

Sama dengan versi Compose, file ini berisi animeList yang menyimpan tujuh objek Anime secara statis menggunakan listOf(). File ini menjadi sumber tunggal data di seluruh aplikasi XML, diakses oleh AnimeViewModel, HomeFragment, dan DetailFragment.

AnimeViewModel.kt (XML)

File ini mengimplementasikan pola MVVM untuk versi XML. AnimeViewModel menerima parameter username bertipe String melalui konstruktor dan menggunakan StateFlow untuk mengekspos data anime secara reaktif. Pada blok init, Timber mencatat dua log: saat user mulai memuat data beserta username, dan saat data berhasil dimuat beserta jumlah itemnya. Ini memastikan data tidak disimpan langsung di Fragment melainkan dikelola oleh ViewModel.

AnimeViewModelFactory berfungsi sebagai pabrik pembuat AnimeViewModel dengan parameter username. Dibutuhkan karena sistem Android tidak mendukung ViewModel dengan konstruktor berparameter secara default. Fungsi create() mengecek tipe ViewModel yang diminta dan mengembalikan instance AnimeViewModel dengan username yang sudah disiapkan, jika tidak cocok maka melempar IllegalArgumentException.

App.kt (XML)

Sama seperti versi Compose, file ini adalah Application Class yang dijalankan pertama kali sebelum komponen lain. Melakukan inisialisasi Timber dengan Timber.plant(Timber.DebugTree()) agar semua Timber.d() di seluruh aplikasi XML

bisa muncul di Logcat. Didaftarkan di AndroidManifest.xml dengan `android:name=".App"`.

HomeFragment.kt (XML)

File ini adalah Fragment utama yang menampilkan daftar anime. Menggunakan `AnimeViewModel` melalui `delegate by viewModels { AnimeViewModelFactory("MuhammadFulan") }`, artinya `ViewModel` dibuat menggunakan `Factory` dengan parameter `username`. Di dalamnya terdapat dua fungsi utama: `setupToolbar()` untuk mengatur `Toolbar` dengan menu bahasa, dan `setupRecyclerView()` untuk membangun `RecyclerView`.

`RecyclerView` menggunakan `ConcatAdapter`, yaitu teknik menggabungkan beberapa adapter menjadi satu list vertikal. `CarouselWrapperAdapter` ditempatkan di atas untuk carousel, dan `AnimeAdapter` di bawahnya untuk daftar anime. Saat tombol `Detail` diklik, `Timber` mencatat log judul anime yang dipilih, lalu `Fragment` mengirim data judul anime melalui `Bundle` ke `DetailFragment` menggunakan `Navigation Component`.

DetailFragment.kt (XML)

File ini menampilkan informasi lengkap satu anime yang dipilih. Menerima data melalui `Bundle` berupa judul anime, kemudian mencari data anime yang cocok dari `animeList` menggunakan `find{}`. Terdapat dua `Timber log` di file ini, pertama saat `Fragment` dibuka mencatat judul anime yang sedang ditampilkan, kedua saat tombol `Back` ditekan juga mencatat judul anime sebelum kembali ke halaman sebelumnya. Gambar dimuat menggunakan library `Glide` dengan efek `centerCrop`.

SettingFragment.kt (XML)

File ini menangani fitur ganti bahasa secara runtime. Menggunakan strategi dua jalur: `LocaleManager` untuk `Android 13` ke atas dan `AppCompatDelegate` untuk versi lebih lama. Berbeda dengan versi `Compose`, `SettingFragment` masih menggunakan `android.util.Log.d()` bawaan `Android` untuk mencatat perubahan bahasa. Setelah

mengganti bahasa, secara eksplisit memanggil `requireActivity().recreate()` untuk memaksa Activity restart dan menerapkan bahasa baru.

AnimeAdapter.kt (XML)

File ini menghubungkan data `List<Anime>` dengan tampilan `item_anime.xml` di `RecyclerView`. Menggunakan pola `ViewHolder` agar view tidak di-inflate berulang kali sehingga lebih efisien. Fungsi `bind()` mengisi setiap view dengan data anime, gambar dimuat menggunakan `Glide`. Tombol Watch mencatat log `Timber` dengan pesan judul anime yang akan dibuka, lalu membuka `Crunchyroll` terlebih dahulu dan fallback ke browser jika tidak terinstall. Tombol Detail meneruskan data anime ke lambda `onDetailClick` yang dihandle oleh `HomeFragment`.

CarouselAdapter.kt (XML)

File ini adalah adapter untuk `ViewPager2` di dalam carousel. Menggunakan teknik `getItemCount() = Int.MAX_VALUE` untuk menciptakan efek infinite scroll. Saat `onBindViewHolder()` dipanggil, index dimodulo dengan ukuran list agar tidak melebihi jumlah data. Gambar carousel dimuat menggunakan `Glide` dengan efek `centerCrop`.

CarouselWrapperAdapter.kt (XML)

File ini adalah adapter pembungkus yang mengemas `ViewPager2` dan dot indicator menjadi satu item di dalam `RecyclerView` utama. `getItemCount()` mengembalikan 1 karena carousel hanya satu blok di dalam `RecyclerView`. Di dalamnya, `ViewPager2` menggunakan `CarouselAdapter` untuk infinite scroll, dan dot indicator diisi secara programatik menggunakan `addView()`. Setiap kali halaman carousel berubah, `updateDots()` dipanggil untuk memperbarui tampilan dot yang aktif.

activity_main.xml

Layout ini adalah container tunggal seluruh aplikasi XML. Hanya berisi satu elemen penting yaitu `FragmentContainerView` yang menjadi wadah tempat semua `Fragment` ditampilkan dan diganti oleh `Navigation Component`. Atribut

`app:navGraph="@navigation/nav_graph"` menghubungkan layout ini dengan graph navigasi yang mendefinisikan semua rute antar Fragment.

fragment_home.xml

Layout layar utama yang sangat sederhana. Berisi Toolbar di bagian atas untuk menampilkan judul aplikasi dan menu, serta RecyclerView yang mengisi sisa ruang layar. RecyclerView inilah yang menampung ConcatAdapter berisi carousel dan daftar anime.

fragment_detail.xml

Layout detail dibungkus dalam ScrollView agar konten bisa digulir saat sinopsis terlalu panjang. Di dalamnya terdapat ImageView berukuran 300dp untuk gambar latar anime, diikuti judul, tahun tayang, label sinopsis, isi sinopsis, dan tombol kembali. Semua elemen teks menggunakan `textAppearance` dari tema Material Design untuk konsistensi tampilan.

fragment_setting.xml

Layout sederhana berbasis LinearLayout vertikal dengan padding 16dp. Berisi Toolbar di atas, label teks pemilihan bahasa, dua tombol bahasa, yaitu bahasa Inggris dan Indonesia, dan tombol kembali. Semua tombol menggunakan lebar `match_parent` untuk tampilan yang konsisten.

item_anime.xml

Template tampilan untuk setiap baris anime di RecyclerView. Menggunakan `MaterialCardView` sebagai container dengan sudut melengkung dan bayangan untuk tampilan yang modern. Layout di dalamnya berbentuk horizontal, gambar anime di sebelah kiri dengan lebar tetap 100dp, dan di sebelah kanan terdapat judul, tahun tayang, deskripsi singkat, serta dua tombol Watch dan Detail menggunakan style `OutlinedButton`.

item_carousel.xml

Template satu slide di dalam ViewPager2. Gambar latar memenuhi seluruh card menggunakan centerCrop. Di bagian bawah terdapat overlay gelap semi transparan untuk meningkatkan keterbacaan teks judul dan tahun tayang yang ditampilkan di atasnya.

item_carousel_wrapper.xml

Layout untuk satu item RecyclerView yang membungkus ViewPager2 dan dot indicator. ViewPager2 dengan tinggi 220dp berada di atas untuk carousel yang bisa digeser, dan LinearLayout dot indicator berada di bawahnya. Keduanya terbungkus dalam satu LinearLayout vertikal sehingga tampil sebagai satu kesatuan blok di RecyclerView utama.

Debugger

Debugger adalah alat untuk membantu developer menemukan dan memperbaiki kesalahan atau bug dalam kode program. Fungsi utama Debugger adalah memungkinkan developer menjalankan program secara terkontrol dengan menghentikan eksekusi program pada titik tertentu yang disebut breakpoint. Saat program berhenti di breakpoint, developer dapat memeriksa nilai variabel, melihat alur eksekusi kode, dan memahami mengapa suatu bagian kode tidak berjalan sesuai harapan. Dengan Debugger, proses identifikasi kesalahan menjadi lebih mudah dan efisien dibandingkan hanya mengandalkan log atau perkiraan semata.

Step Over, Step Into, Step Out

Saat program berhenti di breakpoint, Debugger menyediakan tiga fitur navigasi untuk menelusuri kode secara detail yaitu:

Step Over digunakan untuk menjalankan baris kode saat ini dan melanjutkan ke baris berikutnya tanpa masuk ke dalam fungsi yang dipanggil. Fitur ini cocok digunakan ketika developer sudah yakin bahwa fungsi yang dipanggil di baris tersebut sudah benar dan tidak perlu diperiksa lebih dalam. Contohnya, saat berada di baris

Timber.d("..."), menekan F8 akan langsung melanjutkan ke baris berikutnya tanpa masuk ke dalam cara kerja Timber.

Step Into digunakan untuk masuk ke dalam fungsi yang dipanggil di baris kode saat ini, sehingga developer bisa melihat eksekusi kode di dalam fungsi tersebut secara detail baris per baris. Fitur ini berguna ketika developer ingin memahami cara kerja suatu fungsi atau mencurigai ada bug di dalam fungsi tersebut. Contohnya, saat berada di baris Timber.d("..."), menekan F7 akan membawa developer masuk ke dalam kode internal Timber dan melihat cara Timber memproses pesan log tersebut.

Step Out digunakan untuk keluar dari fungsi yang sedang ditelusuri dan kembali ke kode pemanggil. Fitur ini berguna ketika developer sudah cukup melihat isi fungsi dan ingin kembali ke konteks yang lebih luas. Contohnya, setelah masuk ke dalam kode Timber menggunakan Step Into, menekan Shift+F8 akan membawa developer keluar dari Timber dan kembali ke baris Timber.d("...") di kode semula.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya.

A. Jawaban

Application Class adalah kelas dasar dalam arsitektur aplikasi Android yang dijalankan pertama kali sebelum komponen lain seperti Activity, Fragment, atau Service. Kelas ini merupakan turunan dari `android.app.Application` dan memiliki lifecycle yang mencakup seluruh durasi aplikasi berjalan, mulai dari pertama kali dibuka hingga aplikasi benar-benar ditutup.

Untuk menggunakan Application Class, developer membuat kelas baru yang meng-extend Application, lalu mendaftarkannya di `AndroidManifest.xml` menggunakan atribut `android:name`. Contohnya seperti:

```
class App : Application() {  
    override fun onCreate() {  
        super.onCreate()  
        Timber.plant(Timber.DebugTree())  
    }  
}
```

```
<application  
    android:name=".App"  
    ...>
```

Fungsi Application Class

Pertama, Application Class digunakan sebagai tempat inisialisasi komponen global yang dibutuhkan seluruh aplikasi. Komponen seperti library logging atau Timber, database atau Room, maupun framework dependency injection atau Dagger Hilt/Koin perlu diinisialisasi satu kali sebelum aplikasi berjalan. Dengan meletakkan inisialisasi di Application Class, developer memastikan komponen tersebut siap digunakan di mana saja dalam aplikasi tanpa perlu diinisialisasi ulang.

Kedua, Application Class menjadi tempat ideal untuk menyimpan data atau konfigurasi global yang perlu diakses oleh berbagai bagian aplikasi, seperti konfigurasi API, informasi sesi pengguna, atau pengaturan tema.

Ketiga, Application Class memungkinkan developer memantau status aplikasi secara menyeluruh, seperti saat aplikasi berpindah antara latar belakang dan latar depan, melalui fungsi-fungsi callback yang tersedia.

TAUTAN GITHUB

[Link GitHub Modul 4](#)